

Simulation of Feedback Mechanisms Arising From Co-Evolutions of Diversely Seeded Primordial Stars and Black Holes

Petr Kisselev¹, Eric Xie¹, Tayne Han¹, Shreraj Sainani¹

¹Thomas Jefferson High School for Science and Technology Astronomy and Astrophysics Laboratory

Lab Director: William D. Linch III

June 1, 2026

Abstract

Recent observations of the early universe from data taken by the James Webb Space Telescope have raised new questions on primordial star, galaxy, and supermassive black hole formation in the early universe and have also increased the importance of simulations of conditions in the first billion years after the Big Bang. In particular, recent work has suggested that the formation of the earliest stars, primordial black holes (PBH), and supermassive black holes may influence each other via gravitational interactions and feedback processes, and it is therefore essential to simulate the co-evolution of these entities holistically. In this work, we implement our own grid-based hydrodynamics simulation to model Jeans collapse at small scales, as well as N-body simulations and Smoothed-Particle Hydrodynamics to model star formation at larger scales. After applying curve fitting to a decaying exponential, we found that there was a statistically significant linear and decreasing correlation between the natural log of the fitted amplitude with PBH mass, suggesting that numbers of stars formed is exponentially suppressed by PBH mass. Thus, further investigation is needed to determine if large PBHs were the source of unexpected Population III star formation in the early universe, if such formation possibly occurred despite them, or if such formation was due to lower than expected PBH mass on a larger scale.

1 Introduction

In the traditional scheme of stellar classification, Population III stars correspond to the oldest and most metal-poor stars that were abundant in the early Universe when metal abundance was extremely low. They are no longer detected in the present.

Primordial black holes (PBHs) are theoretical compact black holes formed very early in the history of the Universe (possibly through dark matter density fluctuations) that could be used to explain some of the latest observations by telescopes such as JWST regarding Population III star formation[1]. According to recent observations by JWST, early universe galaxies were detected at redshifts that were higher than expected, indicating that galaxies, and by extension the primordial stars that made up those galaxies, likely formed earlier than existing theoretical models suggested.

The formation of both PBHs and Population III stars are believed to have affected each other. Large Supermassive Black Holes (SMBHs) can draw in large clumps of gas necessary for star formation, influence dark matter structure, and enhance local density fluctuations that help trigger gravitational collapse [2]. They could also, via Active Galactic Nuclei (AGN) energy emissions and tidal heating, disrupt the low temperature and high pressure conditions conducive to stellar formation [3]. Thus, simulating the formation and evolution of those bodies simultaneously is essential to capture such interactions and more accurately model the early universe.

Previous work has attempted to model using simulations the effects of PBHs on the formation of population III stars, the effect of stellar halo mass on SMBH growth, and the co-evolution of galaxies, DM-halos, and SMBHs. [2, 3, 4].

Koulen et. al [2] utilize the open-source GIZMO package [5] to investigate this phenomenon: the effect of PBHs on population III star formation. They employ a hydrodynamical N-body simulation to simulate interactions of “normal” matter and dark matter across kiloparsec-scales, with additional libraries to incorporate initial condition generation and cooling chemistry. The redshift at which the highest gas density value across all particles in the simulations exceeds a critical value is called the “collapse redshift”; by comparing this, the simulation compares the effect differing masses and abundances of PBHs have on the collapse redshift, whether they accelerate (higher z_{col}) or disrupt (lower z_{col}) population III star formation. They find that higher-mass PBHs accelerate star formation regardless of abundance, while lower-mass PBHs do so only at high abundances. However, this work only employs a monochromatic mass distribution, where it assumes that all PBH particles will have the same mass value, and the simulation stops once the critical value for material density is reached, ignoring the feedback subsequent rounds of stellar formation may provide on the gas cloud & the effect of both PBHs and star formation on galactic evolution.

Along similar lines, Li et al. [3] model the co-evolution of SMBHs, galaxies, and dark matter halos using IllustrisTNG [6]. The run is done on a much coarser, megaparsec scale, and thus focuses more on the galactic structure aspect of SMBH growth. The simulation thus covers multiple galactic structures at once. For each run, a “halo” of masses is identified, and the SMBH & total stellar mass of each halo are tracked over time. The paper identified four overarching phases for evolution of numerous galaxies, dominated by star formation, AGN accretion, self-regulation, and mergers, respectively. The paper’s work allows modeling of galactic evolution based on stellar feedback. However, it would be beneficial to identify exactly how the SMBHs form (the simulation simply spawns an SMBH once the halo mass exceeds a certain value). In addition, due to the large scale of the simulation, information provided by each run on the morphological evolution of each galaxy may be comparatively limited.

In this work, we implement our own, computationally optimized simulation to simulate the gravitational and pressure-based motions of galactic gas and stars, and how the evolution of a square region of a galaxy with sides 260 light years wide depends on a central PBH mass.

2 Analytic Methods

We implement a Smoothed-Particle Hydrodynamics (SPH) [7] approach in C++, which overall is an approximation for the hydrodynamic equation for force density.

Notably, it is a Lagrangian simulation method, meaning that the computation of physical quantities is associated with points that move in space over time, as opposed to a fixed point in space as traditional Eulerian methods do. As is common practice, we represent PBHs, stars, and even gas clouds as points (“particles” hereafter) and evolve/compute properties for them over time, including ones usually not defined for a single particle (such as density and pressure). Essentially, implementing SPH allows us to apply the fluid dynamics equations of a traditional, grid-based cell to moving particles representing physical bodies with less expensive computation.

2.1 Gravitational Interactions

One of the components on the accelerations of all particles is the gravitational force. We model the system as a two-dimensional N -body simulation under Newtonian gravity. Each particle represents a fluid element, and evolves under the gravitational influence of all other fluid particles.

The magnitude of the force between two particles of masses m_i and m_j , separated by a distance $r_{ij} = |\mathbf{r}_j - \mathbf{r}_i|$, is given by a softened gravitational law:

$$F_{ij} = \frac{Gm_i m_j}{(r_{ij} + \epsilon)^2}$$

where ϵ is a softening parameter introduced to prevent numerical divergences at small separations and to approximate finite spatial resolution.

The net force on each particle is computed by summing pairwise interactions over all other particles. The net force $\mathbf{F}_i$ for particle with mass m_i is:

$$\mathbf{F}_i = \sum_{j \neq i}^N F_{ij} \frac{\mathbf{r}_j - \mathbf{r}_i}{\epsilon + r_{ij}}$$

Accelerations are obtained via Newton’s second law, where each particle’s acceleration is $\mathbf{g}_i = \frac{\mathbf{F}_i}{m_i}$. This framework provides the gravitational backbone of the simulation, which is later coupled with hydrodynamic modeling to capture gas dynamics and star formation processes.

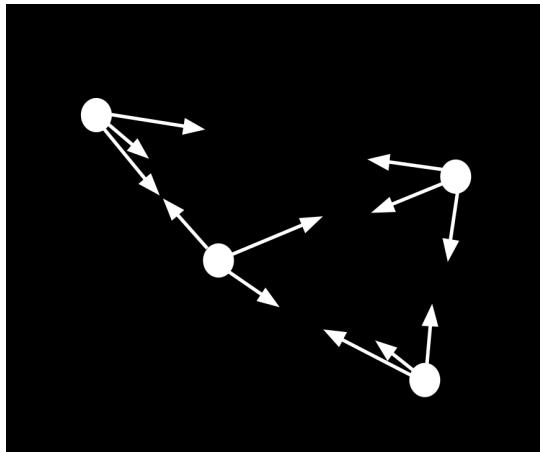


Figure 1: Forces shown between four particles.

However, this approach when used on a large scale, being a naive one, overall contains inefficiencies which are resolved in Section 3. Two examples would be that this approach is recomputing forces already given by Newton’s Third Law, and that it is $O(N^2)$

2.2 The Kernel

As aforementioned, the fluid is represented by discrete particles that carry mass, velocity, and thermodynamic properties [7, 8]. In this framework, fluid quantities are obtained by smoothing over neighboring particles, using a kernel weighting function.

Generally, a kernel $W(|\mathbf{r}_i - \mathbf{r}_j|, h)$ is used to weigh contributions from nearby particles when calculating properties. It approximately resembles a gaussian distribution in shape, weighing particles located close to the particle more heavily. Parameter h is the *smoothing length* of the kernel, and characterizes how ”narrow” the kernel centered around the central particle is. In our simulation, h takes on a different value for each particle, based on how close they’re located with respect to other particles.

We use the 2-D version of the Wendland C2 kernel:

$$W_{ij} = A(1 + 2q)(1 - \frac{q}{2})^4,$$

where $q = \frac{r}{h}$ is the normalized distance between the two particles the kernel is working on and A is a normalization constant. Note that the function equals 0 at $q = 2$; we truncate the kernel to be 0 beyond $q = 2$, effectively ignoring particles located too far apart when computing the weighted sum for the properties. With this constraint, one can find that the normalization constant equals $A = \frac{7}{4\pi h^2}$.

Analogous to how we use the probability density function to integrate and obtain the probability, we can deduce that the kernel W has dimensions of inverse area. This can also be seen from the $\propto h^{-2}$ dependence of the normalization constant.

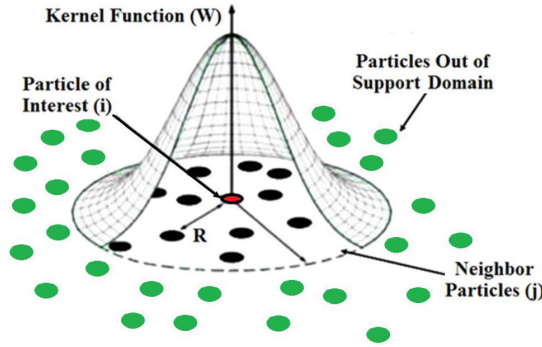


Figure 2: Visualization of kernel used in SPH with smoothing length R . [9]

2.3 Smoothed-Particle Hydrodynamics

The main equation of motion for SPH simulations is

$$\frac{d\mathbf{v}_i}{dt} = -\frac{\nabla p}{\rho} + \mathbf{g},$$

with $\mathbf{g}$ as described in Section 2.1. To update the motions of the particles, we thus need to calculate the particles’ densities and pressure gradients at each time step using a discretized method.

To find the ”density” of the particle, we utilize the total weighted sum of masses within a certain distance from the particle. Thus, the local density at particle i is computed as:

$$\rho_i = \sum_{j \neq i}^N m_j W(|\mathbf{r}_i - \mathbf{r}_j|, h),$$

where W is a smoothing kernel with characteristic length scale h , and the sum runs over neighboring particles.

Pressure is then computed assuming isothermal conditions, as $p = \frac{\rho RT}{\mu}$.

We then use a discretized version of the ∇p operator to obtain that the equation of motion is equal to

$$\frac{d\mathbf{v}_i}{dt} = - \sum_{j \neq i}^N m_j \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2} \right) \nabla W(|\mathbf{r}_i - \mathbf{r}_j|, h) + \frac{1}{m_i} \sum_{j \neq i}^N F_{ij} \frac{\mathbf{r}_j - \mathbf{r}_i}{\epsilon + r_{ij}},$$

when we consider gravitation in addition to hydrodynamics as well, where p_i and ρ_i are the pressure and density at particle i .

As can be seen, hydrodynamic effects are coupled with gravitational effects in this acceleration. Positions and velocities are then advanced in time based on the resulting acceleration using numerical integration. To ensure long-term stability and approximate conservation of energy, we employ the Leapfrog integration scheme below.

$$\begin{aligned} \mathbf{v}_{n+1/2} &= \mathbf{v}_{n-1/2} + \mathbf{a}_n dt \\ \mathbf{x}_{n+1} &= \mathbf{x}_n + \mathbf{v}_{n+1/2} dt \end{aligned}$$

This symplectic method updates velocities and positions in a staggered manner, reducing secular energy drift and making it well-suited for gravitational systems. This coupling of SPH with gravitational dynamics enables modeling of the evolution of baryonic matter alongside primordial black holes. This allows gas to collapse under gravity, redistribute through pressure forces, and form dense regions that can seed star formation. The combination of SPH and N -body gravity enables one to capture both hydrodynamic behavior and gravitational feedback within a unified framework based on particles.

2.4 Star Formation

The main condition for star formation is the exceeding of the Jeans Critical Density, derived from the following dispersion relation for pressure waves in a molecular cloud [10]:

$$\omega^2 = c_0^2 k^2 - 4\pi G \rho_0$$

Gas clouds become unstable to collapse when ω becomes imaginary, implying exponentially growing pressure and density wave amplitudes, so Jeans collapse occurs when the right side of the above equation is negative. The critical density of a gas cloud is thus:

$$\rho_c = \frac{c_0^2 k^2}{4\pi G} = \frac{RT}{4\pi\mu G} \left(\frac{2\pi}{\lambda} \right)^2$$

where we substituted the isothermal speed of sound $c_0 = \sqrt{\frac{RT}{\mu}}$ and usual formula for wave number $k = \frac{2\pi}{\lambda}$ [10]. The wavelength is assumed to be the smoothing length of the gas cloud, so the critical density, which we employed in our simulation is:

$$\rho_c = \frac{\pi}{Gh^2} \left(\frac{RT}{\mu} \right)$$

where the term in parentheses is the same constant we used to convert density to pressure in our calculation of the pressure gradients, since the ideal gas law states $p = \frac{RT}{\mu} \rho$.

In every time step of our calculation, for each gas particle, our simulation checked the Jeans condition, which would require every neighboring particle in the surrounding kernel to have a density exceeding the

Jeans density. In that case, our simulation spawned in a star, and since the timescale of star formation is typically much shorter than the overall lifetime of a star [11], we chose to instantaneously initialize stars.

To test our numerical methods such as leapfrog integration, we simulated in python the Jeans collapse of a molecular cloud on a smaller scale and higher resolution compared to our main simulation (see Fig. 3), which confirmed the accuracy of both these equations for Jeans collapse and of leapfrog integration methods. As shown in the figure, in which blue corresponds to regions of high density, as the simulation progressed, disturbances in pressure originating in a small region spread out and the resulting density and pressure waves grew in an unstable manner, resulting in a rapid collapse of the surrounding gas cloud.

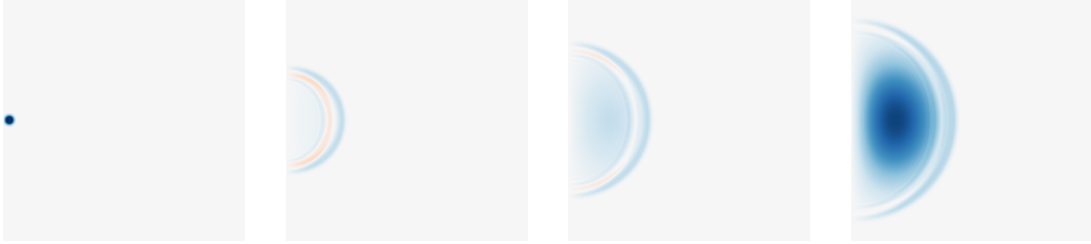


Figure 3: Snapshots of Jeans Instability simulation coded from scratch in python in chronological order, with left image as the initial frame and right-most image as the final simulation frame.

3 Computational Methods

3.1 Introduction

As mentioned earlier, the “naive” approach to simulating some number of discrete particles n involves computing the force vector through a direct summation of all other bodies.

$$\mathbf{F}_i = \sum_{j \neq i}^N F_{ij} \frac{\mathbf{r}_j - \mathbf{r}_i}{\epsilon + r_{ij}}$$

This approach entails a computational time complexity of $O(N^2)$. This quadratic time complexity arises from the reality that each particle will need to compute through every other, and this scaling quickly destroys any chances for a simulation implemented through this approach being able to handle more than $10^3 - 10^4$ bodies under reasonable time constraints. Some limited optimizations can be made, such as caching the force computation to be mirrored when considering the opposite orientation between two bodies, cutting computation in half. Such optimizations offer limited constant-time benefits but ultimately do not solve the scaling problem at hand. For that, a new approach is needed.

3.2 Tree-based Methods

In order to approach the problem of computational complexity, it is necessary to fundamentally restructure the approach that is used. Rather than considering particles to behave effectively as some unstructured list, implementing a spatial structuring of the particles would enable greater insight into the computation that could be leveraged for efficiency gain. In the case of the n -body problem, the historical strategy chosen is the use of a quadtree or octree (Barnes-Hut)[12].

We chose to implement our simulation to represent a two-dimensional “slice” of a three-dimensional structure. As a result, we maintain the behavior of forces as in standard three-dimensional space while avoiding the additional complexity of visualization and the higher fidelity needed. As a result, the quadtree structure becomes the choice that is natural for our simulation.

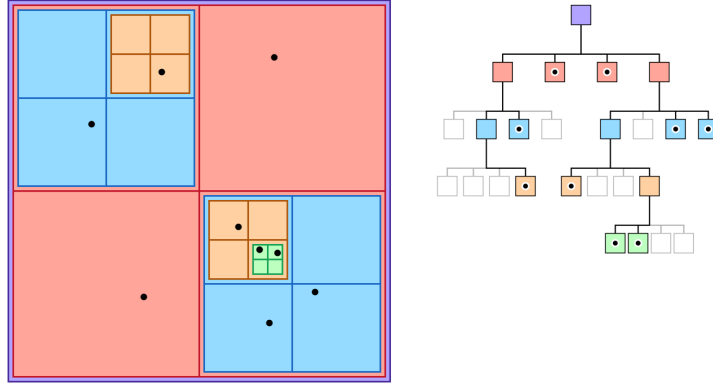


Figure 4: Example of quadtree-based spatial subdivision [13].

3.3 Barnes-Hut

The Barnes-Hut approach mentioned earlier is critical to enabling high-fidelity simulations at scales that can support the large structural phenomena observed in galactic dynamics. The algorithm operates by recursively subdividing space into quadrants via the quadtree structure described above, grouping spatially proximate particles into hierarchical clusters. This structure can be seen in Figure 4.

Rather than computing the gravitational contribution of every individual body, a given particle i instead interacts with an internal node of the tree, treating the cluster it represents as a single pseudo-particle located at its center of mass, provided that the cluster is sufficiently distant. This condition is formalized through the Barnes-Hut criterion. For a given internal node representing a region of width s at a distance d from particle i , the node is treated as a single body if

$$\frac{s}{d} < \theta$$

where θ is the “opening angle” parameter which defines accuracy. While $\theta = 0$ recovers the direct summation exactly, higher values enable farther-away bodies to be “clumped” and approximated by center of mass. When the criterion is not satisfied, the algorithm descends further into the node’s children, repeating the evaluation recursively until either the criterion is met or a leaf node containing a single particle is reached.

This structure reduces the force calculation for a single particle from an $O(N)$ traversal of all bodies to an $O(\log N)$ traversal of the tree, as the depth of the quadtree scales logarithmically with the number of particles. Summing over all N particles, the Barnes-Hut algorithm achieves an overall time complexity of $O(N \log N)$, a dramatic improvement over the $O(N^2)$ direct summation.

3.4 Quadtree Parallelization

While Barnes-Hut is a powerful method that enables N -body computation of many bodies ($10^4 - 10^5$), approaching 10^6 bodies simulated requires more specific optimizations. The most direct and important, is parallelizing the quadtree build and the acceleration computation.

For tree construction, the chaotic nature of the simulation means that it becomes difficult to predict how work must be shared in order to avoid memory contention while also maximizing threading to get the best overall performance. Simply splitting up the particles into sections that are worked on individually by threads does not work, however, as this approach necessarily result in the different independent threads conflicting as a result of trying to write to the same areas. Additionally, there is significant cost inherent in assigning threads to too many different sections as a result of scheduling overhead. The simulation code exploits the structure of the tree itself, by recursively dispatching computation of subtrees, which are then distributed across available threads. This is also combined with check that ensure the necessity of such a split and thread synchronization. This adaptive choice enables the code to take advantage of

both the traditional benefits of threading speedup while avoiding overhead and opting for concurrent programming when necessary.

The acceleration tree-walk that is central to the Barnes-Hut algorithm can also benefit from a threading speedup. Here, thread contention is a non-issue as gravitational acceleration is computed per body and therefore can be done completely independently. A simple schema of assigning different bodies to different threads can work well, but such an approach tends to leave some threads “waiting” for work as their bodies tend to be clustered in sparser regions, where the quadtree doesn’t need to split as deep. In order to assign bodies more evenly, the simulation keeps track of the total number of quadtree nodes that each thread visits on each timestep, and shifts the assignment of bodies to even out the “work” on the next one. This approach works as a result of the basic principle that between two time steps, particle are unlikely to change position enough to move from one thread’s region of work to another.

3.5 Computational SPH

While the previous methods were sufficient to implement the gravitational attraction of particles, forces associated with Smoothed Particle Hydrodynamics require additional computation. In particular, the kernel function $W(r, h)$ (Wendland C2), has compact support, meaning that it falls to zero beyond $2h$ and that only particles within the smoothing radius h contribute meaningfully to the evaluation. As a result, determining the value of h is critical to accurately computing the value of density and pressure at each particle. The practical challenge is that while a higher value of h can be better, it can also introduce regions of varying density within this radius. The simplest approach that can take advantage of the existing quadtree structure would be one that ties the value of h to some multiple of the bounding quadtree node’ size, as it is expected to be roughly proportional to the density. This is a “quick and dirty” approach that however leaves a lot on the table in terms of accuracy. In reality, smoothing length is solved for iteratively. A good value of h is one that encloses sufficient mass to give a stable estimate to density, formalized through the condition $h \propto \sqrt{\frac{m}{\rho}}$. As ρ or density requires a value of h to be computed, h is iteratively solved for through this condition at each particle until the value stabilizes [14].

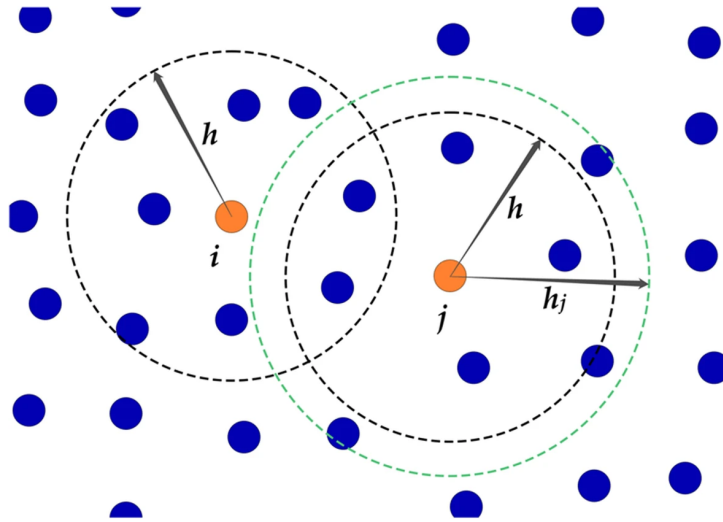


Figure 5: Example of adaptive smoothing length [14].

Now in order to compute density itself, it is necessary to find all particles that surround a given particle within the radius $2h$. This is inherently an expensive operation as the structure of the quadtree does not inherently lend itself to distance-based queries of neighboring particles. Querying all particles within $2h$ at every particle, at every time step is naive and wasteful: fluid particles tend to move smoothly and continuously so the set of neighbors a particle has at one time step is likely to be almost the same set even a few time steps later. This can be exploited by tracking a neighbor list for each particle that is reused across multiple steps. Furthermore, to maintain accuracy with this new approach, a skin radius can be implemented. Rather than only recording particles that fall within the kernel support radius of

$2h$, the neighbor list is built with an enlarged search radius that pads the kernel boundary. This means that between neighbor list updates, particles that might move out of the kernel support radius can be replaced by nearby particles that move in. Once enough particles have crossed out of this “skin” buffer, the neighbor list is rebuilt, before the accuracy can drop. The actual neighbor list construction still needs to be efficient, however. The simulation performs a targeted quadtree traversal that performs a distance test at each internal node of the tree to check if its minimum distance to the given particle is outside the radius and can therefore be skipped. When reaching a node that can not be skipped, only particles that pass the padded radius $(2 + \delta_{skin})h$ are admitted into the built neighbor list. Stars and PBHs are excluded as they are point-like and do not contribute to the density or pressure of the fluid, instead acting gravitationally. With this list of neighbors, density is computed through the kernel method explained earlier by weighting masses through the function and normalizing for the weighting. Pressure is similarly computed through the earlier expressed relationship between pressure and density that can be found under isothermal conditions, which is an exact relation.

3.6 Data Layout and Memory Access Patterns

Beyond the algorithmic complexity, the practical performance of the N -body SPH simulation is highly dependent on how well its memory access patterns mesh with the cache hierarchy of modern processors. A CPU cache line fetches a fixed block of contiguous memory regardless of how much is needed, meaning that the specific layout of particle data in memory directly impacts how much actually usable information is retrieved with every cache lookup and how much bandwidth needs to be wasted every cache miss. Furthermore, modern CPUs can operate so fast that waiting for a cache lookup can cost hundreds of CPU cycles spent idling instead of computing. Scaled across $N = 10^6$ bodies, this cost adds up.

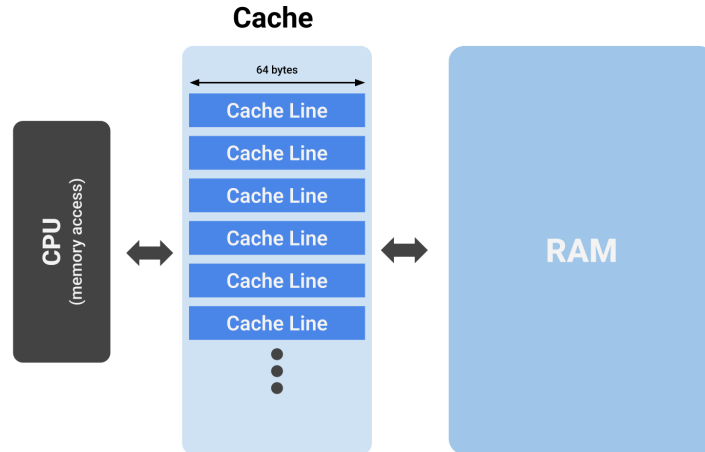


Figure 6: Visual representation of the memory pipeline.

The particle data in the simulation is organized as a structure of arrays (SoA) as opposed to the more intuitive array of structures (AoS) layout. While this may seem like a minor difference it has big implications for the cache usage of the simulation. In an AoS layout, every time that a particle needs to be referenced, everything about it fetched all at once: position, velocity, mass, density, acceleration, etc. This is convenient from the perspective of legibility and the object oriented approach, but extremely harmful for the access patterns that arise from SPH. Density computation only touches position and mass while finding pressure requires velocity and density as well. Instead of dragging every particle’s entire structure of information through the limited cache, SoA enables the simulation to only access that which is necessary for the current computation, grouping properties in contiguous arrays. Cache lines can be optimally saturated with information to ensure that the CPU threads don’t need to wait on the cache to catch up.

The throughput of cache can be further improved through an explicit reordering of particles after every tree rebuild. After construction of the tree, particles are stored in the same order they appear in a depth-first traversal of the tree. This ordering ensures that spatially contiguous sections of particles are

likely to occupy nearby positions in memory. During the construction of the neighbor lists for SPH, this becomes particularly important as the values that the tree-walk needs to query are close in memory, avoiding expensive lookups as the probability that a needed value was already loaded in a previous cache line is dramatically increased. This benefit can be applied to Barnes-Hut itself, by enabling the leaves of the quadtree to store more than one particle. While this leads to more computation, it allows leaf particles to arrive together in memory. Considering nodes in the quadtree to store particles as range-based indices in the depth-based hierarchy instead of specific references to individual particles allows the full effectiveness of the introduced spacial cache locality to take effect.

3.7 Simulation Overview

For each run, a central, massive PBH is spawned, with fluid particles randomly distributed throughout the galactic region, and later on, with stars spawning throughout the simulation when the density exceeds the Jeans critical density for star formation at specific particles where all surrounding particles in the kernel have met that condition as well. When this occurs, the mass of all particles within that kernel surrounding a particle become halved for the purposes of mass conservation. The mass of the resulting star is the resulting mass of the other halves of the masses of the particles. The kernel smoothing length h adapts to keep the number of particles in the kernel constant, which of course depends on the particle in question. Importantly, only the fluid gas particles will experience the ∇p component on its acceleration; i.e., for stars and PBHs, the equations of motion will simply be $\frac{d\mathbf{v}_i}{dt} = \mathbf{g}$. Simulations were ran with central PBH mass as the independent variable and the number of stars formed as the dependent variable on a time-scale of 1000 frames, or around 33 real-time seconds.

To further simplify the calculations performed by our code and increase the readability of our source code, we performed nondimensionalization on length, time, and mass. We applied these unit conversions, ultimately allowing us to use the values $G = 1$ and $m_{\text{PBH}} \sim 1$. In addition, we chose parameters N and m_p . This effectively determined our other values listed in Table 1, which we calculated using existing known constants, such as the approximate mass of population III stars.

Simulations operated under the following nondimensionalized parameters:

Parameter	Value	SI
G	1	$6.67 \times 10^{-11} \text{ kg}^{-1} \text{ m}^3 \text{ s}^{-2}$
m_{PBH}	1, 5, 10, 15...80	$\{1, 5..\} \times 10^{34} \text{ kg}$
m_p	0.0005	$5.0 \times 10^{30} \text{ kg}$
m_S	0.1	$1.0 \times 10^{33} \text{ kg}$
N	100000	100000
L_{sim}	24	260 ly
T	2.7	2.7 K

Table 1: Simulation parameters, where T is the constant temperature, m_p is the mass of a fluid particle, m_{PBH} is the mass of the central primordial black hole, m_S is the mass of stars, and L_{sim} is the side length of the square region of simulation. Nondimensionalized units are used here with their SI equivalents on the right.

4 Results

When implementing SPH to update the densities of each of the stars, we observed star formation (new stars colored in green) occurring along regions of high densities, which were the filaments marked in yellow and red (see Fig. 7).

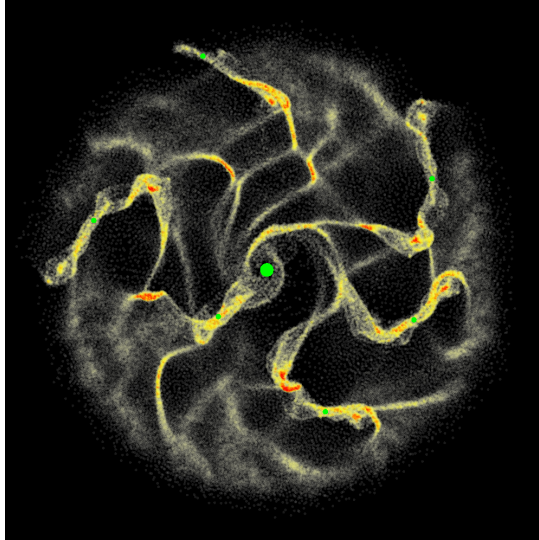


Figure 7: A depiction of the halo and gas particles & stars shortly after star formation begins.

In our initial trial, we allowed star formation to occur immediately after the simulation began. However, this resulted in star formation occurring exclusively at the outer edges of our galaxy, which was unrealistic because these are regions of relatively low density (Fig. 8). The reason for this unexpected result is likely due to the initial gas particles having random velocities, and since there was insufficient time for pressure forces to allow particles to spread out, some regions had abnormally high densities compared to others, which resulted in unrealistic rapid star formation. Thus, in all subsequent simulations, we added a delay time, in which the density and pressure would continue to evolve but no star formation was allowed to take place. This effectively allowed gas pressure forces to smooth out the noise in the initialized simulation. In the following simulations, as soon as the delay time had passed, we observed star formation taking place at slower rates, and primarily in regions with high density, as expected.

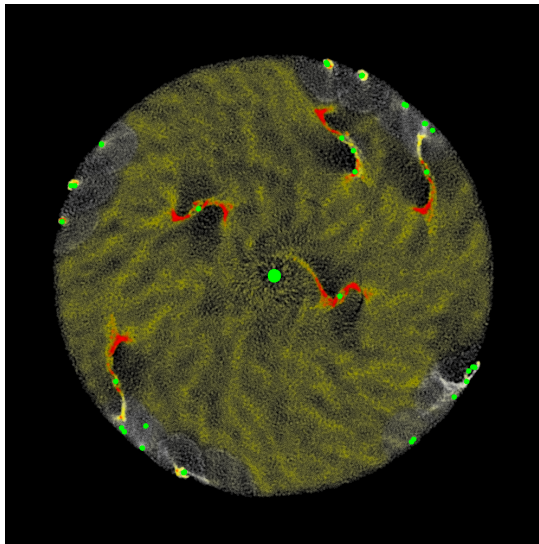


Figure 8: Results without the initial delay time.

When adjusting the mass of the central PBH, we observed a difference in the structure of the resulting star formation and gas particles. For a central body with a large mass, star formation occurred primarily in dense regions close to the center (Fig. 9), likely due to the high densities resulting from the large gravitational fields caused by the central PBH. On the other hand, when there was no significant central body mass, star formation occurred in isolated patches, some of which were farther from the center, and in an asymmetrical pattern (Fig. 10).

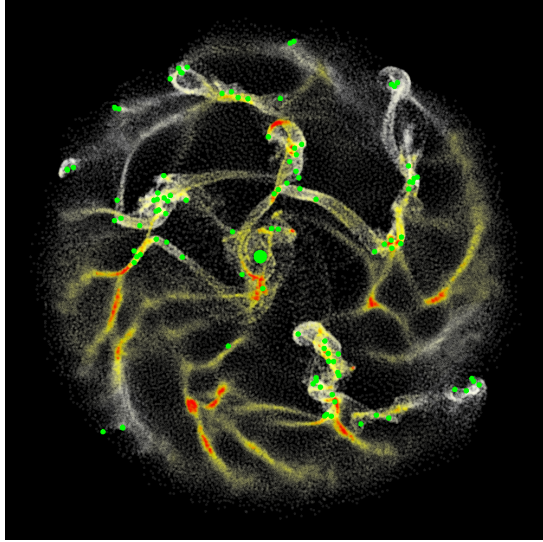


Figure 9: Same as Figure 7, but with slightly different smoothing calculations.

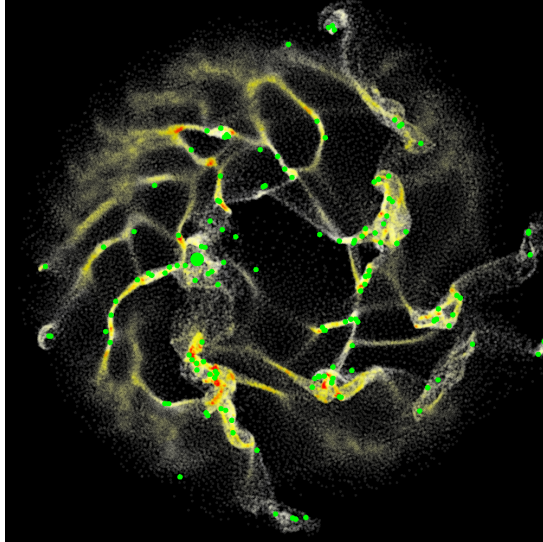


Figure 10: Results with a lower central PBH mass than other runs.

For trials involving different initialized central PBH masses, we recorded numbers of stars formed at each time step, and our results are plotted in Fig. 11. On the horizontal axis is the simulation time in seconds after the delay on star formation expired. According to the figure, the first few stars were formed at around the same time in the simulation, but the rate of star formation and final number of stars formed varies greatly, with higher PBH masses (shown in yellow) associated with a lower number of stars formed, and lower masses (shown in purple) associated with more primordial star formation.

Towards the end of the simulation, the star formation rate levels off and appears to approach a constant value. The curves of star count over time appears to be a decreasing exponential of the form:

$$N = A(1 - e^{-k(t-t_0)})$$

where t_0 is the time of formation of the first star, which we estimated to be 2 seconds, while A and k are constants determined by applying least squares curve fitting on these curves. We extracted the constant A , which represents the number of stars the simulation tends towards, and we plotted the natural log of A over PBH mass (Fig. 12).

In the plot, the point corresponding to $M = 30$ was omitted due to inaccurate results from least squares curve fitting and thus an unrealistic value of A . The plot with the remaining data points follows an

approximately linear pattern, and the line of best fit given by a linear regression analysis appears as a red dashed line. The result of the significance test yielded a p-value of $p = 2.45 \cdot 10^{-8}$, which is lower than our significance level $\alpha = 0.01$, indicating that there was a significant correlation between PBH mass and the natural log of amplitude. The regression coefficient was $r = -0.9478$. We can thus model final star count with the equations:

$$\ln A = aM + b$$

$$A = Ce^{aM}$$

with $a = -0.0362$ and $b = 7.180$. The line of best fit in Fig. 12 ultimately continued to accurately follow star count as central PBH mass was increased over a factor of 80.

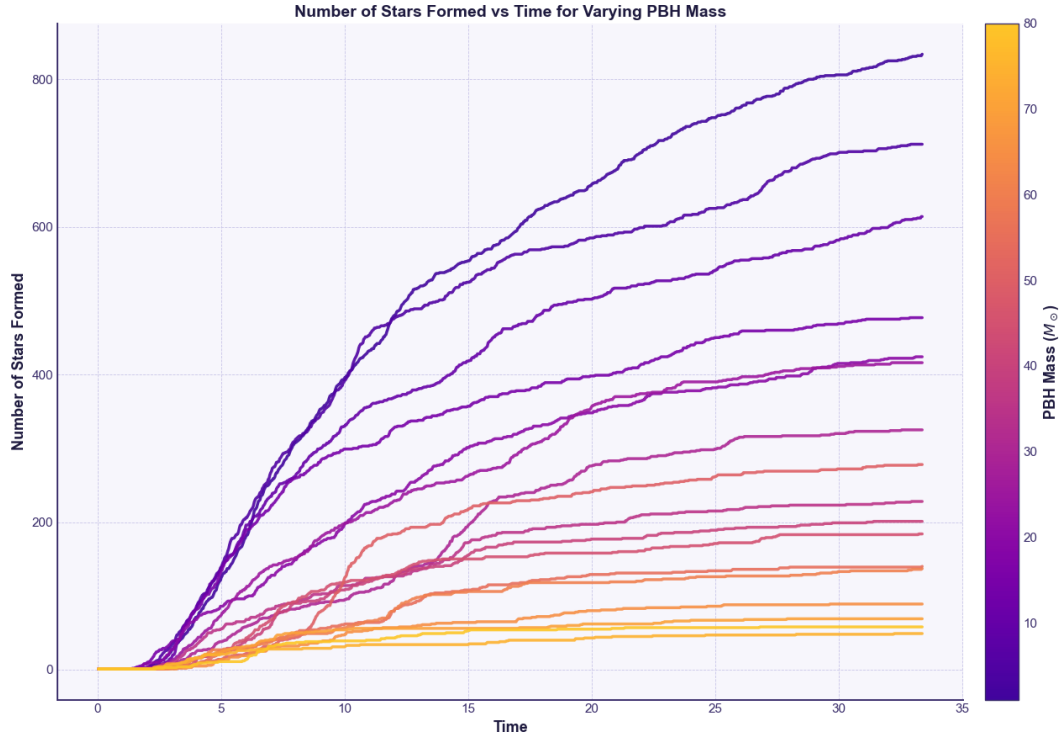


Figure 11: Number of stars formed in simulation over nondimensionalized time for different PBH masses.

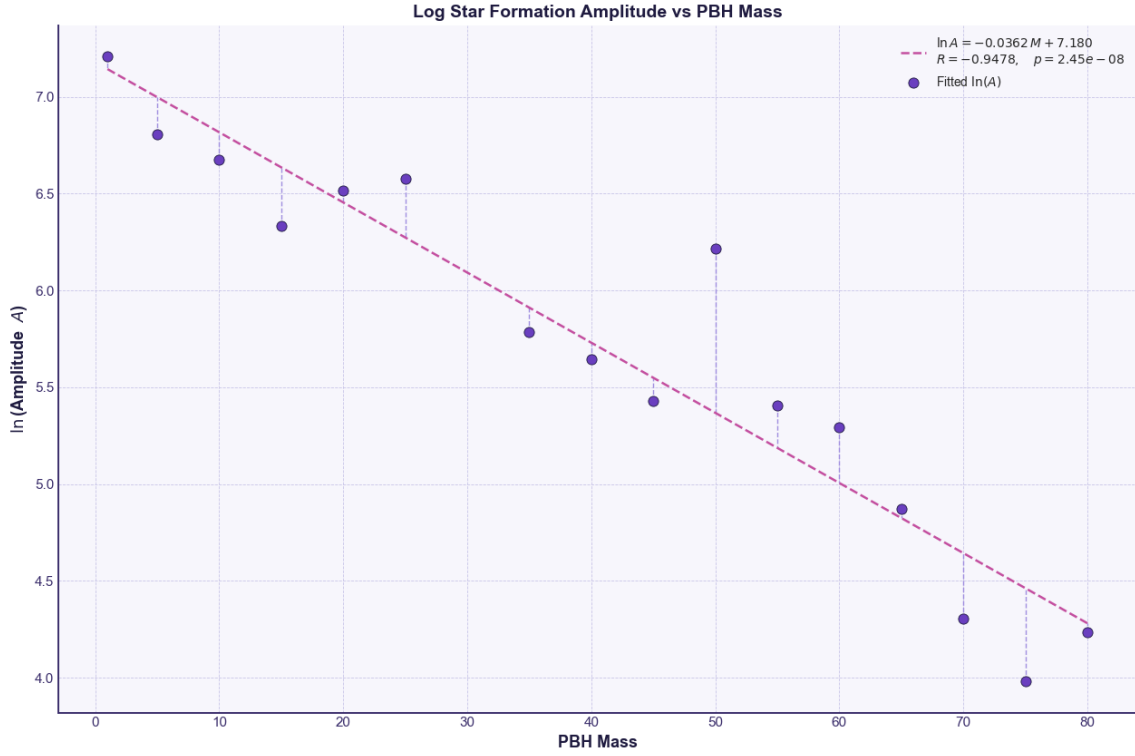


Figure 12: Number of stars formed in simulation over nondimensionalized time for different PBH masses.

5 Discussion

We observed star formation primarily occurring in regions of high densities, marked in yellow and red in our simulation. This likely occurred because star formation requires the density of a gas cloud to exceed the Jeans density, which is approximately constant throughout our simulation, so star formation occurred in regions with highest density first.

We observed that the rate of star formation was heavily dependent on the given critical Jeans density, which is the density required for star formation to take place. When we increased the critical density by a factor of 10, we found that no star formation took place. When decreasing the critical density by a factor of 10 from the original, all gas particles were quickly converted to stars. This indicates that whether or not star formation can occur depends heavily on the value of the Jeans density, which implies that modeling star formation continues to be a useful tool for testing models of density and mass distributions in the early universe.

We noticed a difference in gas cloud structure between simulations with different PBH masses. In the low PBH mass simulation, the resulting stars and gas particles followed an asymmetric pattern, and the central PBH drifted towards one side of the simulation, which was due to the mass of the gas clouds and stars being more significant in the simulation which allowed the PBH to drift. At the same time, filaments also formed throughout the simulated region, including far from the PBH, for both the high and low PBH mass cases, but we found that regions typically involving star formation differed in the two cases. When PBH mass was low, star formation occurred both near the PBH and in filaments far away, but for the high mass case, star formation was suppressed close to the center of the simulation, likely due to tidal effects which would have also suppressed clumping of the gas clouds, which would have been more significant in the high PBH case. Although our simulation accounts for tidal disturbances on the gas cloud, we did not account for tidal heating, which is believed to be another mechanism through

which PBH prevents primordial star formation.

When we began data collection of star count over time, the number of stars formed in the first two seconds of the simulation remains near zero, suggesting that star formation rate starts at near zero but accelerates after a few stars have already been formed. Later on in the simulation, star count levels off as the gas particles tend to no longer possess enough mass to form clumps in regions with high enough Jeans density to undergo molecular gas cloud collapse. As illustrated in Fig. 11, star formation roughly follows a decaying exponential, but random variations in star count and star formation still occur, which in the case of some PBH masses $M = 30$ and $M = 50$, results in outliers in the A value calculated via curve fitting. We chose to omit the $M = 30$ data point from our regression analysis but not the other outlier point, though a future study would ideally run many trials for each PBH mass and average out the resulting star count, which should reduce the impact of simulation instabilities without removing any data points.

In Fig. 12, the residuals of the data points from the line of best fit appear randomly scattered about the line with no clear pattern, indicating that this plot is linear and confirming that a linear regression analysis is appropriate for this data. We found a significant negative correlation between $\ln A$ and PBH mass, so larger PBHs ultimately reduced in lower star formation rates likely through mechanisms such as tidal disturbances and greater gravitational ejection of gas particles. The amplitude of the fitted decaying exponential, which represents predicted star count after a long simulation runtime, is proportional to e^{aM} , and since the value of a given by regression analysis is negative, this is a decaying exponential. In the plot, the horizontal axis is the PBH mass, with no natural logs applied. If A was associated with a power law function of M in the form:

$$A \propto M^{-k}$$

Then $\ln M$ and $\ln A$ in the following equation are linearly related:

$$\ln A = -k \ln M + c$$

which would have resulted in the points in Fig. 12 displaying a clear deviation from the straight line with the same pattern as an exponentially decaying function. Since such patterns for the residuals did not appear, the star formation count thus follows an exponentially decaying function with respect to M , rather than a simple power law. These results further strengthen the claim that large PBHs very strongly discourage the formation of primordial stars under early universe conditions, although future studies may explore this relationship over a wider range of PBH mass and over several orders of magnitude to verify whether not the exponential relation continues to hold.

6 Conclusions and Future Work

In this study, we implemented a cosmological simulation integrating elements of both gravitational N-body and fluid hydrodynamics methods to simulate population III star formation in a region with length scale $L \sim 100$ light years. We find that the rate of formation of stars, which is very sensitive to both the Jeans collapse density and the mass of the PBH, decreases with increasing PBH mass. In particular, we find that the star formation rate is well-fitted by a decaying exponential function, whose amplitude itself has a negative exponential relationship with respect to the mass of the central PBH, showing that increasing central PBH mass significantly suppresses stellar formation.

Several explanations exist for why higher-mass PBHs could disrupt star formation. Tidal disruptions, in which forces act on a gas cloud differentially & apply shear force, would be stronger if the mass of the central PBH was higher. This would decrease the density of the gas cloud, and make it more difficult for the cloud to collapse. Alternatively, more massive central PBHs could make it easier for it to gravitationally eject less massive stars and/or stellar cloud material. Our work suggests that PBHs are likely not a cause of the JWST observations specified in Section 1, which claim that galaxy formation occurred at earlier times than previously suggested. For that reason, our findings demonstrate that more research into mechanisms of star and galaxy formation in the early universe is needed to refine existing cosmological models.

Aspects of our work that could be improved upon in the future includes, firstly, inclusion of more PBH bodies and a variable mass distribution. Other work sometimes includes a larger number of smaller PBHs in addition to the central body, usually with all PBHs having a constant mass in any given run (the distribution is then called *monochromatic*). Future work could implement satellite PBHs with non-monochromatic mass distributions to see how the results may be affected.

In addition, our simulation currently does not include any direct radiative feedback from high-energy radiation emitted by the star, which would ionize the surrounding gas cloud and change the pressure equation of state, which future work could choose to improve upon.

Finally, most galactic-scale bodies have dark matter halos extending beyond the visible portion of the galaxy. Future work could incorporate a similar, extended DM halo that extends beyond the visible, "star-forming material" portion of our simulation.

7 Acknowledgments

We benefited from various discussions with Dr. William D. Linch III, on topics ranging from galactic spiral arm formation to general relativity.

8 Source Code

The simulation source code can be found at <https://github.com/pkissele/physicSim>.

References

- [1] Boyuan Liu and Volker Bromm. Accelerating early massive galaxy formation with primordial black holes. *The Astrophysical Journal Letters*, 937(2):L30, September 2022.
- [2] Julia Monika Koulen, Stefano Profumo, and Nolan Smyth. Primordial black holes and the first stars. *Physical Review D*, 2025.
- [3] Hao Li, Yangyao Chen, Huiyuan Wang, and Houjun Mo. Physical processes behind the co-evolution of haloes, galaxies, and supermassive black holes in the illustriatng simulation. *Monthly Notices of the Royal Astronomical Society*, 543(2):1878–1898, September 2025.
- [4] Lewis R Prole, John A Regan, Daxal Mehta, Peter Coles, and Pratika Dayal. Primordial black holes in cosmological simulations: growth prospects for supermassive black holes. *The Open Journal of Astrophysics*, 8, August 2025.
- [5] Philip F. Hopkins. A new class of accurate, mesh-free hydrodynamic simulation methods. *Monthly Notices of the Royal Astronomical Society*, 450(1):53–110, April 2015.
- [6] Federico Marinacci, Mark Vogelsberger, Rüdiger Pakmor, Paul Torrey, Volker Springel, Lars Hernquist, Dylan Nelson, Rainer Weinberger, Annalisa Pillepich, Jill Naiman, and Shy Genel. First results from the illustriatng simulations: radio haloes and magnetic fields. *Monthly Notices of the Royal Astronomical Society*, August 2018.
- [7] J. J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and Astrophysics*, 30:543–574, 1992.

- [8] Daniel J. Price. Smoothed particle hydrodynamics and magnetohydrodynamics. *Journal of Computational Physics*, 231:759–794, 2012.
- [9] Mohammadreza Bagheri, Masoud Mohammadi, and Masoud Riazi. A review of smoothed particle hydrodynamics, 2024.
- [10] Benny Lautrup. *Physics of continuous matter : exotic and everyday phenomena in the macroscopic world*. Taylor Francis, 2011.
- [11] J M Diederik Kruijssen, Andreas Schruba, Melanie Chevance, Steven N Longmore, Alexander P S Hygate, Daniel T Haydon, Anna F Macleod, Julianne J Dalcanton, Linda J Tacconi, and Ewine F van Dishoeck. Fast and inefficient star formation due to short-lived molecular clouds and rapid feedback. *Nature*, 569, 2019.
- [12] Josh Barnes and Piet Hut. A hierarchical $O(n \log n)$ force-calculation algorithm. *Nature*, 324(6096):446–449, Dec 1986.
- [13] Apple Inc. Gkquadtree, 2026. Apple Developer Documentation.
- [14] Min Li, Hongshu Li, Weiliang Meng, Jian Zhu, and Gary Zhang. An efficient non-iterative smoothed particle hydrodynamics fluid simulation method with variable smoothing length. *Visual Computing for Industry, Biomedicine, and Art*, 6, 2023.